MobileLLM: Comparing Deep-Thin vs Wide-Shallow Architectures for Small Language Models

Yaliang Cai, Chun Zhou, Haobo Zu
Lund University, Sweden

Abstract

Recent research, such as MobileLLM, suggests that "deep-thin" architectures are optimal for sub-billion parameter language models. This work re-examines this design philosophy at the extremely small scale of 60M parameters. We first established a reproduction pipeline using 80M models on RedPajama, and subsequently conducted a systematic comparison of 60M architectures trained on FineWeb-Edu. Contrary to prevailing scaling laws (Hoffmann et al., 2022), our empirical results demonstrate that "wide-shallow" architectures (12–16 layers) tend to outperform "deep-thin" counterparts (30–43 layers) on average at this scale, particularly in reading comprehension tasks. Additionally, we validate the usability of these micro-models by demonstrating that they can acquire instruction-following capabilities through LoRA fine-tuning, albeit with constrained reasoning depth.

1 Introduction

Large language models (LLMs) have achieved impressive performance on many language understanding and generation tasks, but their computational and memory costs make on-device deployment difficult. MobileLLM Liu et al. (2024) studies how to design sub-billion-parameter models that run efficiently on mobile or edge hardware, and reports that **deep-thin** architectures (more layers with smaller hidden dimensions) work better than **wide-shallow** ones at a fixed parameter budget. In this project, we revisit this depth–width trade-off in the 60M–80M "micro-model" regime under much tighter data and compute constraints. We first build an 80M reproduction of the MobileLLM setup on RedPajama, and then train a family of 60M models on FineWeb-Edu, varying only the allocation of parameters between depth and width. Our results show that wide-shallow architectures (12–16 layers) tend to achieve higher average zero-shot accuracy than deep-thin counterparts (30–43 layers) on commonsense benchmarks such as BoolQ and PIQA, while a 60M base model can still be converted into a simple instruction-following model via LoRA fine-tuning.

2 Related Work

2.1 MobileLLM

MobileLLM (Liu et al., 2024) studies how to design sub-billion-parameter language models that are suitable for on-device deployment. Building on the LLaMA architecture, it keeps RoPE positional encodings, RMSNorm, SwiGLU-style feed-forward networks and grouped-query attention (GQA) (Ainslie et al., 2023), while introducing parameter-saving techniques such as layer sharing (Lan et al., 2019) and shared input–output embeddings. The models are trained from scratch on high-quality RedPajama-v1 corpora, including ArXiv and Wikipedia. A key empirical finding is that, at a fixed parameter budget, **deep-thin** architectures (more layers with smaller hidden size) consistently outperform **wide-shallow** architectures (fewer layers with larger hidden size) on downstream benchmarks.

Our work follows the same general design recipe, but scales the models down to about 60M and 80–95M parameters and trains them within a single-GPU budget. We use publicly available corpora and substantially fewer training tokens, and re-examine this depth–width trade-off in a much smaller training setup.

2.2 Architecture Design for Small Language Models

Work on small language models (Team et al., 2024) (Zhang et al., 2024) has explored a range of architectural knobs, including network depth, hidden width, feed-forward expansion ratio, attention head allocation and parameter sharing. For a fixed pa-

rameter budget, increasing depth can improve expressivity by stacking more non-linear transformations, while increasing width gives each layer more representational capacity but reduces the number of layers that can be afforded. Recent LLaMA-style models typically combine RoPE positional encodings, RMSNorm, SwiGLU feed-forward networks and multi-head attention as standard building blocks, and then adjust depth and width to target different parameter scales. In addition, grouped-query attention, layer sharing and shared input–output embeddings are commonly used to reduce memory footprint without changing the overall computation pattern.

Our work follows this design philosophy. Rather than introducing new components, we keep the same LLaMA/MobileLLM-style blocks and focus on the trade-off between depth and width under a fixed parameter budget. This allows us to isolate how the depth–width allocation affects downstream performance at 60M and 80M scales, without conflating it with changes in tokenizer, context window or normalisation scheme.

3 Methodology

3.1 Dataset Construction

We use different pre-training corpora for the 80M and 60M model families, but keep the same tokenizer and sequence format across all experiments. In both stages, raw text is tokenized with the MobileLLM LLaMA tokenizer (vocabulary size 32,000; maximum sequence length 2,048). We insert a beginning-of-sequence (BOS) token at the start of each packed sequence and an end-of-sequence (EOS) token at document boundaries when available. Using the HuggingFace `datasets` library with `streaming=True`, we iterate over sharded JSONL files without loading them entirely into memory. The tokenized text is then packed into fixed-length 2,048-token sequences. We do not maintain a separate pre-training validation set. Instead, we periodically save checkpoints during training and later evaluate selected checkpoints on downstream benchmarks.

3.1.1 Pre-training corpus for the 80M models

For the initial 80M experiment, which mainly serves to validate our training and evaluation pipeline, we follow the corpus choice of MobileLLM and build a pre-training dataset from RedPajama-v1 (Weber et al., 2024). We select two well-structured and semantically dense domains, ArXiv and Wikipedia, and read them as sharded JSONL files. Each record is tokenized with the MobileLLM tokenizer (using `<s>` as BOS and `</s>` as EOS), and the resulting token streams from ArXiv and Wikipedia are interleaved and packed into sequences of length 2,048. The processed corpus occupies roughly 60 GB on disk and is used exclusively for training the 80M deep and wide models.

3.1.2 Pre-training corpus for the 60M models

For the main 60M-scale experiments, we switch to the FineWeb-Edu dataset (Lozhkov et al., 2024)(Gunasekar et al., 2023), using the smallest released configuration (`sample-100BT`). FineWeb-Edu is a filtered subset of the larger FineWeb corpus, derived from CommonCrawl and curated to emphasise educational and informational content. In our setup, the raw text amounts to slightly more than 60 GB before tokenization. We process it with the same LLaMA tokenizer and packing strategy as above: documents are streamed from disk, tokenized with BOS/EOS markers, and packed into fixed-length 2,048-token sequences. The resulting packed dataset occupies on the order of 100 GB on disk. All four 60M models (Deep, Wide, Extreme Deep, Extreme Wide) share this pre-training corpus; differences in performance therefore stem from architectural and training choices rather than data.

3.2 Experimental Setup

3.2.1 Common architectural and training settings

All models in our study follow the same LLaMA-style, decoder-only Transformer architecture as MobileLLM. Concretely, we use causal self-attention with Rotary Positional Encodings (RoPE), RMSNorm layers, and SwiGLU-style feed-forward networks (implemented as `silu` in the HuggingFace configuration). To reduce the parameter count without changing the computation pattern, we enable grouped-query attention (GQA) with a smaller number of key–value heads, layer sharing across blocks, and shared input–output token embeddings. The vocabulary size is fixed to 32,000 and the maximum sequence length to 2,048 for all experiments.

We also keep the optimisation setup identical across all 80M and 60M models. Training is performed on NVIDIA A100 GPUs with mixed-

precision arithmetic (`bf16`). We use the AdamW optimiser with $\beta_1 = 0.9$, $\beta_2 = 0.95$ and $\epsilon = 10^{-8}$, a cosine learning-rate schedule with a peak learning rate of 1.5×10^{-4} and 2,000 warmup steps, and a weight decay of 0.1. The global batch size is fixed to 64 sequences in all runs; for the 80M models this is realised as 2 GPUs with per-device batch size 8 and gradient accumulation over 4 steps, while for the 60M models we use a single GPU with per-device batch size 8 and accumulation over 8 steps. Gradient checkpointing is enabled for the deeper 60M variants to reduce memory usage, and disabled otherwise.

3.2.2 80M model architectures

As an initial sanity check of our training and evaluation pipeline, we design two models at roughly 80M parameters by scaling down the MobileLLM-125M configuration. Both models keep the common settings described above (tokenizer, RoPE positional encodings, RMSNorm, SwiGLU-style feed-forward networks and grouped-query attention), and differ only in their depth–width allocation.

The **80M Deep** model increases the number of Transformer blocks while using a relatively small hidden size. It has 28 layers, a hidden dimension of 480, an intermediate (feed-forward) dimension of 1,280, and 10 attention heads with 2 key–value heads. This corresponds to a clear *deep-thin* design.

The **80M Wide** model trades depth for width. It uses 16 layers with a hidden size of 640 and an intermediate dimension of 1,920, while keeping the same 10 attention heads and 2 key–value heads. This yields a shallower but wider network with higher per-layer capacity, i.e. a *shallow-wide* design. Because of the larger hidden and feed-forward dimensions, this configuration has a higher total parameter count (95.20M versus 82.47M for 80M Deep), although we still refer to both models as "80M" baselines. This initial 80M study is mainly used to validate our data pipeline and training setup, so we do not enforce strict parameter parity between the deep and wide variants. The main architectural choices are summarised in Table 1.

Table 1: Architectural configuration of the 80M models.

Configuration	80M Deep	80M Wide
Layers	28	16
Hidden Size	480	640
Intermediate Size	1280	1920
Attention Heads	10	10
KV Heads	2	2
Total Parameters	82.47M	95.20M
Depth/Width (L/d_{model})	0.058	0.025

3.2.3 60M model architectures

Our main study focuses on four models around 60M parameters, constructed to probe both MobileLLM-style depth–width ratios and more extreme configurations. All four models share the same tokenizer, sequence length, attention block design and optimisation settings; only the number of layers, hidden size, intermediate size and attention head allocation are changed. In what follows, we refer to them as **60M Extreme Deep**, **60M Deep**, **60M Extreme Wide** and **60M Wide**.

The **Deep** and **Wide** variants are scaled versions of the original MobileLLM deep and wide baselines. The Deep model uses 30 layers with a hidden size of 384 and an intermediate dimension of 1,024, while the Wide model uses 12 layers with a hidden size of 576 and an intermediate dimension of 1,536. Both keep grouped-query attention with a small number of key–value heads (2 and 3 respectively), and lie close to the 60M-parameter target.

The **Extreme Deep** and **Extreme Wide** variants push this trade-off further while staying in the same parameter range. Extreme Deep increases the depth to 43 layers with a thinner hidden size of 320 and an intermediate size of 960, using 5 attention heads and a single key–value head, corresponding to a very *deep-thin* design. Extreme Wide, in contrast, reduces the depth to 6 layers but widens the hidden size to 768 and the intermediate size to 1,920, with 12 attention heads and 4 key–value heads, representing a very *shallow-wide* configuration. These extreme designs allow us to test whether very deep or very wide networks are more parameter-efficient at this scale. The full configurations are given in Table 2.

Table 2: Architectural configuration of the 60M models.

Configuration	60M Extreme Deep	60M Deep	60M Extreme Wide	60M Wide
Layers	43	30	6	12
Hidden Size	320	384	768	576
Intermediate Size	960	1024	1920	1536
Attention Heads	5	6	12	9
KV Heads	1	2	4	3
Total Parameters	60.46M	59.50M	60.56M	60.91M
Depth/Width (L/d_{model})	0.134	0.078	0.008	0.021

3.3 Evaluation Protocol

To ensure rigorous and reproducible assessment of the model architectures, we implemented a standardized evaluation pipeline focusing on zero-shot commonsense reasoning capabilities.

3.3.1 Methodology: Zero-Shot Checkpoint Selection

Given the strict token budget and the goal of maximizing training efficiency, we adopted a "posterior selection" strategy rather than reserving a static validation set during pre-training.

- **Framework:** We utilized the industry-standard `lm-evaluation-harness` library to perform automated evaluations. All assessments were conducted in `float16` precision to match the training environment.
- **Checkpoint Strategy:** Instead of relying on validation loss (perplexity) which may not perfectly correlate with downstream capability, we saved model checkpoints periodically. We then evaluated these checkpoints directly on downstream tasks to identify the optimal iteration.
- **Metric:** We report **Zero-shot Accuracy** for all tasks. Zero-shot evaluation is particularly stringent for models at the 60M scale, as it tests the raw architectural capability to understand and complete tasks without the aid of few-shot examples.

3.3.2 Benchmark Suite: Commonsense Reasoning

We selected a diverse set of standard benchmarks to probe different aspects of the models' reasoning and knowledge retrieval capabilities:

BoolQ (Boolean Questions) (Clark et al., 2019): A reading comprehension dataset derived from Google queries. It tests the model's ability to process a passage and answer binary (Yes/No) questions. This serves as a primary proxy for the model's language understanding and context retention.

PIQA (Physical Interaction QA) (Bisk et al., 2020): Focuses on physical commonsense reasoning (e.g., knowing that "to cool a glass, you put it in the fridge," not "the oven"). This tests whether the architecture has captured grounded knowledge about real-world object interactions.

HellaSwag (Zellers et al., 2019): A challenging dataset for commonsense natural language inference (NLI), requiring the model to select the most plausible ending to a context. It is notoriously difficult for small models and serves as a stress test for complex inference capabilities.

ARC-easy (AI2 Reasoning Challenge): Consists of grade-school science questions. We included this task (specifically for the 60M main study) to evaluate the models' ability to recall scientific facts and perform simple reasoning steps.

4 Results

4.1 Evaluation on zero-shot commonsense benchmarks

For the 80M experiment, we evaluate the 80M Deep model at checkpoint 40k and the 80M Wide model at checkpoint 60k, corresponding to the last saved checkpoints of each run. All evaluations are performed in float16 precision with automatic batch-size selection on a single A100 GPU. We report accuracy as the primary metric for all three tasks and compute the average performance across tasks to compare the overall effectiveness of the two architectural approaches.

Table 3 presents the evaluation results for the 80M architectures on the three downstream tasks. The deep-thin model achieves an average accuracy of 43.49% across all tasks, while the wide-shallow model obtains an average of 46.15%.

Table 3: 80M Model Downstream task evaluation results. Accuracy (%) on zero-shot commonsense reasoning benchmarks.

Model	BoolQ	PIQA	HellaSwag	Avg.
Deep (40k)	52.14	52.72	25.70	43.49
Wide (60k)	61.77	51.31	25.36	46.15

Table 4: 60M model downstream task evaluation results. Accuracy (%) on zero-shot commonsense reasoning benchmarks.

Model	BoolQ	PIQA	HellaSwag	ARC-easy	Avg.
60M Extreme Deep	52.39	48.86	**26.31**	25.93	38.37
60M Deep	46.94	48.91	25.84	**26.73**	37.11
60M Extreme Wide	52.17	**51.36**	25.82	25.17	38.63
60M Wide	**60.61**	51.31	26.15	25.88	**40.99**

Table 4 summarises the 60M-scale results. Among the four variants, **60M Wide** achieves the best average accuracy (40.99%), driven mainly by its strong performance on BoolQ (60.61%). **60M Extreme Wide** attains the highest score on PIQA

(51.36%), while **60M Extreme Deep** slightly outperforms the others on HellaSwag (26.31%) and **60M Deep** leads on ARC-easy (26.73%). Overall, the two wide models dominate on BoolQ and PIQA, whereas the deep models remain competitive on the more challenging HellaSwag and ARC-easy benchmarks.

4.2 Findings

Based on the empirical results presented in Tables 3 and 4, we distill three key findings regarding the architectural scaling laws at the 60M and 80M parameter regime.

Reversal of Scaling Laws at Micro-Scale. Contrary to the "deep-thin" preference observed in sub-billion models (e.g., MobileLLM at 125M+), our experiments suggest a reversal of this rule at the 60M limit. As shown in the Phase 1 validation (Table 3), the **80M Wide-Shallow** model outperforms its deep-thin counterpart by **2.66%** in average accuracy, driven by a massive **+9.63%** gain in BoolQ. This trend is confirmed in the 60M main study (Table 4), where the **60M Wide** variant achieves the highest performance on BoolQ (60.61%), and the two wide models are strongest on PIQA (with **60M Extreme Wide** reaching 51.36%), significantly surpassing both Deep variants. This suggests that when parameter budget is extremely scarce, prioritizing width (feature dimension) over depth preserves essential representational capacity for reading comprehension.

The "Sweet Spot" of Depth. While wider models generally performed better, "extreme" shallowness proved detrimental. Comparing the two wide variants in the 60M group (Table 3), the **60M Wide** (12 layers) significantly outperformed the **60M Extreme Wide** (6 layers) on BoolQ (60.61% vs. 52.17%). This indicates that a minimum depth threshold is required to form complex semantic dependencies. A 6-layer network appears insufficient for language understanding tasks, whereas 12 layers strikes an optimal balance between trainability and reasoning depth at this scale.

Task-Specific Inductive Biases. Performance gains were not uniform across tasks. The advantage of Wide architectures is predominantly concentrated in **BoolQ** (Reading Comprehension) and **PIQA** (Physical Interaction). However, for tasks requiring complex inference chains or scientific knowledge recall, Deep models remained competitive. Notably, the **60M Extreme Deep** (43 layers) achieved the highest score on HellaSwag (**26.31%**), and the **60M Deep** led on ARC-easy (**26.73%**). This suggests that while Width aids in general context processing, extreme Depth may still offer marginal benefits for abstract inference, albeit at the cost of training stability and generalizability in QA tasks.

5 Instruction Fine-tuning of MobileLLM-60M via LoRA

5.1 Overview

This experiment explores the potential of micro-scale language models (approximately 60 million parameters) in Instruction Following tasks. We selected the `MobileLLM-60M-Extreme-Deep` as the Base Model. Utilizing the Unsloth (Han et al., 2023) optimization framework and LoRA (Low-Rank Adaptation) technology, we performed Supervised Fine-Tuning (SFT) on the Alpaca-Cleaned dataset. The primary objective was to transform a base model, which originally possessed only text-continuation capabilities, into an instruct model capable of understanding user intent and conducting basic single-turn question answering.

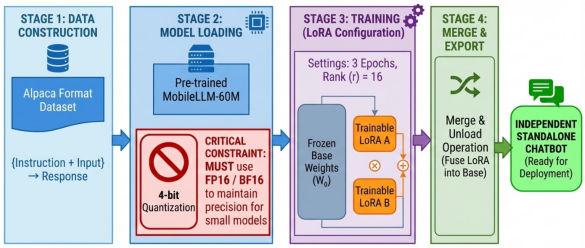


Figure 1: The technical implementation pipeline for efficient instruction tuning of MobileLLM-60M. The process details data formatting (Stage 1), precision constraints during loading (Stage 2), LoRA-based training configuration (Stage 3), and final model merging (Stage 4).

5.2 Experimental Setup

We utilized the 60M Extreme Deep variant. Unlike standard wide architectures, this model employs a "deep-thin" design (43 layers, 320 hidden size) to maximize abstract reasoning capabilities within a constrained parameter budget. No quantization (`load_in_4bit=False`) was applied to preserve weight precision. The dataset employed was `yahma/alpaca-cleaned` (approx. 51,760 pairs), using the standard Alpaca prompt template:

```
### Instruction: {instruction}
### Input: {input}
### Response: {output}
```

Fine-tuning Strategy We adopted LoRA with the following configuration: Rank (r) = 16, Alpha (α) = 16. All linear layers were targeted (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) to maximize the efficacy of low-rank adaptation without altering the pre-trained weights.

Model Parameters Details Total Parameters: 62,753,664 (approx. 62.75M). Trainable Parameters: 3,256,320 (approx. 3.26M). Training Ratio: 5.19% (typical efficient parameter fine-tuning settings).

Hyperparameters The training was accelerated using Unsloth's Triton kernels. A larger batch size was permitted due to the small model size to stabilize gradient descent. The detailed hyperparameters are listed in Table 5.

Table 5: Hyperparameters for MobileLLM-60M Fine-tuning

Hyperparameter	Value
Batch Size	32
Learning Rate	2×10^{-4}
Epochs	3 (Total steps: 4,854)
Optimizer	AdamW (8-bit)
Scheduler	Linear (Warmup steps: 10)
Max Sequence Length	2048
Precision	Float16 (Mixed Precision)

5.3 Results and Analysis

Training Convergence The training process lasted approximately 80 minutes. According to the WandB logs, the loss function showed a stable downward trend, indicating effective learning of the instruction format. The loss decreased from an initial value of $\approx$ 13.2 (at Step 30) to a final value of 3.0.

Model Merging and Export Upon completion, we performed a Weight Merging operation to fuse the LoRA adapter weights into the base model using the `model.merge_and_unload()` method. This step ensured the model could operate independently as a standalone Instruct Model. The final model was saved in `safetensors` format and uploaded to the Hugging Face hub.

Inference Optimization Due to the limited capacity of the 60M parameter model, preliminary tests showed a tendency towards hallucination and repetition loops. To mitigate these issues during inference, we implemented the following generation constraints:

1. Repetition Penalty: Set to 1.2 to forcefully penalize repeated tokens.
2. Temperature: Lowered to 0.1 to reduce randomness and ensure conservative, coherent responses.
3. Length Constraint: `max_new_tokens` limited to 64.
4. Post-processing: Manual truncation to remove self-generated tags.

5.4 Conclusion

This experiment demonstrates that through three epochs of fine-tuning, a 60M parameter Base Model can be successfully transformed into an Instruct Model. While the model's reasoning depth is limited by its size, it successfully acquired the ability to follow instruction formats and conduct basic single-turn Q&A, validating the efficacy of the "deep-thin" architecture combined with PEFT methods.

6 Discussion

6.1 Divergence from Scaling Laws at the Extreme Scale

Our primary finding—that wide-shallow architectures outperform deep-thin ones at the 60M and 80M scale—stands in contrast to the conclusions of the original MobileLLM study, which advocated for depth in sub-billion parameter models. We hypothesize that this divergence stems from a critical "width threshold" inherent to transformer architectures. In the MobileLLM study (focusing on 125M+ models), the thinest models likely maintained a sufficient hidden dimension to propagate feature representations effectively. However, in our `60M-Extreme-Deep` configuration, the hidden dimension is constricted to 320. We argue that at such an extreme constriction, the width becomes an information bottleneck. Regardless of how deep the network is (43 layers), the limited channel capacity prevents the model from retaining sufficient information from the input tokens, thereby degrading performance in knowledge-intensive tasks like reading comprehension. This suggests that scaling laws are not linear; below a certain parameter

threshold (likely ≈100M), preserving a minimum width takes precedence over adding depth.

6.2 The Optimization Landscape of Micro-Models

The superior performance of the `Wide` (12 layers) over the `Deep` (30 layers) also points to optimization challenges specific to training from scratch on limited compute. Deep networks are notoriously harder to optimize due to gradient vanishing/exploding issues, even with normalization techniques like RMSNorm. In our Phase 1 validation, we observed that the Wide model peaked later in training steps (60k) compared to the Deep model (40k) but achieved a higher final accuracy. This implies that wider, shallower networks may offer a smoother optimization landscape for small-scale pre-training, allowing the model to continue learning effectively for longer durations without plateauing prematurely.

6.3 Alignment vs. Reasoning: The Limits of SFT

Our instruction tuning experiments reveal a distinct dichotomy in the capabilities of 60M models. The model successfully learned to follow interaction formats (e.g., stopping generation, adopting an assistant persona), proving that *alignment* is achievable even with minimal capacity. However, the persistence of hallucinations in logic-heavy tasks (e.g., symbol manipulation) indicates that SFT operates primarily on surface-level stylistic distributions rather than inducing robust reasoning circuits. This aligns with the "surface form competition" hypothesis: micro-models can memorize the *form* of an answer but lack the parameter budget to build the deep abstraction hierarchies required for complex reasoning. Therefore, while 60M models are viable for simple, retrieval-based instruction following, they require strict inference engineering (e.g., low temperature, repetition penalties) to mask their inherent instability.

7 Future Work

While this study establishes the architectural superiority of wide-shallow models at the 60M parameter scale, several avenues remain for enhancing the practical utility and performance of such micro-LLMs.

7.1 Knowledge Distillation for Reasoning

Our results indicate that while 60M models can learn surface-level alignment through SFT, they struggle with complex reasoning tasks. To address this, future research will focus on Knowledge Distillation (KD). Instead of training from scratch using only hard labels (next-token prediction), we plan to utilize a larger "Teacher" model (e.g., LLaMA-3-8B) to guide the 60M "Student." By minimizing the KL divergence between the teacher's and student's probability distributions, we aim to transfer the reasoning patterns and "soft knowledge" of the larger model to the compact architecture, potentially bridging the performance gap in logic-intensive benchmarks like HellaSwag.

7.2 On-Device Deployment and Quantization

The ultimate goal of the MobileLLM project is deployment on resource-constrained edge devices. We intend to investigate post-training quantization techniques, specifically reducing model precision from `float16` to `int4` or `int8`. We will evaluate the trade-offs between precision loss and inference latency on real-world hardware, such as Raspberry Pi and mobile processors (e.g., Snapdragon or Apple A-series chips). This will validate whether our wide-shallow architectures maintain their performance advantage under aggressive quantization and strictly limited memory bandwidth.

7.3 Scaling Data and Training Duration

Our current experiments were conducted on a subset of FineWeb-Edu due to compute constraints. However, "Chinchilla" scaling laws suggest that 60M models are far from being over-trained. Future work will explore training on significantly larger datasets (e.g., 1T+ tokens) to determine the saturation point of these architectures. We hypothesize that with prolonged training, the capacity gap between Deep and Wide models might manifest differently, potentially revealing new insights into the long-term convergence behaviors of micro-scale Transformers.

8 Conclusion

In this work, we revisited the architectural design choices for efficient Large Language Models at the extreme scale of 60M parameters, challenging the prevailing "deep-thin" consensus established for sub-billion models. Through rigorous pre-training on RedPajama and FineWeb-Edu datasets under a

constrained GPU budget, we derived the following conclusions:

First, we identify a reversal of scaling laws in the sub-100M regime. Our empirical results demonstrate that **wide-shallow architectures (12–16 layers)** consistently outperform deep-thin counterparts (30–43 layers) on zero-shot commonsense benchmarks. Specifically, the 12-layer Wide model achieved a 13.6% advantage in reading comprehension (BoolQ) over the Deep model, suggesting that maintaining a sufficient hidden dimension is critical for representational capacity when the total parameter budget is strictly limited.

Second, we determined an architectural "sweet spot." While width is beneficial, extreme shallowness (e.g., 6 layers) degrades performance, indicating that a moderate depth is still necessary to form semantic dependencies. The 12-layer configuration emerged as the optimal balance point between trainability and reasoning capability at the 60M scale.

Finally, we demonstrated the usability of these micro-models through Low-Rank Adaptation (LoRA). We showed that a 60M model can successfully acquire instruction-following behaviors and alignment with human intent ("Chat Mode"). However, our qualitative analysis highlights that while surface-level alignment is achievable, robust reasoning remains constrained by model size, necessitating strict inference engineering to mitigate hallucinations.

Future work will focus on enhancing the reasoning capabilities of these wide-shallow models through knowledge distillation from larger teacher models (e.g., LLaMA-3) and exploring 4-bit quantization for real-world deployment on mobile edge devices.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*.

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, and 1 others. 2020. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2924–2936. Association for Computational Linguistics.

Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, and 1 others. 2023. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*.

Daniel Han, Michael Han, and Unsloth team. 2023. Unsloth.

Jordan Hoffmann and 1 others. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.

Zhenzhong Lan and 1 others. 2019. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*.

Zechun Liu and 1 others. 2024. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. *arXiv preprint arXiv:2402.14905*.

Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. 2024. Fineweb-edu: The finest collection of educational content the web has to offer. `https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu`.

Gemma Team and 1 others. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Maurice Weber, Daniel Y. Fu, Quentin Anthony, Yonatan Oren, Shane Adams, Anton Alexandrov, Xiaozhong Lyu, Huu Nguyen, Xiaozhe Yao, Virginia Adams, Ben Athiwaratkun, Rahul Chalamala, Kezhen Chen, Max Ryabinin, Tri Dao, Percy Liang, Christopher Ré, Irina Rish, and Ce Zhang. 2024. Redpajama: an open dataset for training large language models. *NeurIPS Datasets and Benchmarks Track*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800. Association for Computational Linguistics.

Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*.